LABORATÓRIO DE SISTEMAS EMBARCADOS & COMUNICAÇÃO

DEPARTAMENTO DE ENGENHARIA ELETRÔNICA

SISTEMAS CIBERFÍSICOS E INTERNET DAS COISAS

GUIA TÉCNICO DE SISTEMAS EMBARCADOS & IOT

Microcontroladores ESP32, Automação Residencial e Redes TCP/IP

Domínio: IoT & Redes de Comunicação — **Plataforma:** ESP32 (ESP-IDF / Arduino Core)

Autor: João Victor Pariz

Engenharia Eletrônica – Outubro de 2026

Visão Geral da Arquitetura do Sistema

Este projeto consolida a integração entre a teoria de redes de computadores e o desenvolvimento prático de sistemas ciberfísicos embarcados:

1. **Fase 1 – Programação com Sockets em C:** Desenvolvimento de arquiteturas cliente-servidor orientadas a conexão (TCP) e sem conexão (UDP) utilizando a API de Sockets BSD / Winsock2.
2. **Fase 2 – Internet das Coisas e Laboratório ESP32:** Desenvolvimento prático de firmware em C++ para microcontroladores ESP32 integrando automação de um **módulo de 4 relés para controle de cargas em corrente alternada** via Wi-Fi (Servidor Web HTTP com SoftAP de contingência) e Bluetooth Classic (SPP).

Este manual consolida o roteiro técnico passo a passo para a bancada: instalação de ferramentas, pinagem segura, circuito elétrico, códigos-fonte e resolução de falhas.

1 O Kit de Automação Residencial Eletrogate

O ecossistema experimental baseia-se na plataforma ESP32 integrada a módulos de controle de potência e instrumentação de ambiente. Os principais recursos utilizados incluem:

- **Kit de Automação Eletrogate:** `https://www.eletrogate.com/kit-arduino`
- **Apostila Oficial de Automação:** Apostila Eletrogate (Automação Residencial)
- **Componentes Chave do Laboratório:**
  1. Placa de desenvolvimento **ESP32 DevKit V1 (30 pinos)**.
  2. Módulo de **4 relés eletromecânicos** de 5V com acionamento óptico por optoacoplador.
  3. Sensor de temperatura e umidade **DHT11**.
  4. Lâmpadas / cargas piloto para simulação dos 4 cômodos residenciais (Sala, Quarto, Cozinha, Varanda).
  5. Cabos de conexão (Jumpers macho-macho e macho-fêmea).

2 Instalação e Configuração do Ambiente de Software

A preparação do ambiente de desenvolvimento deve garantir suporte completo ao ecossistema ESP-IDF / Arduino-ESP32 Core. São necessários os seguintes passos:

2.1 Passo 1: Pré-requisito Obrigatório – Instalação do Python

Nota Técnica sobre Dependências do Toolchain Python

Antes de configurar a placa na IDE, é indispensável que o **Python 3.x** esteja instalado no Windows com a opção **"Add Python to PATH"** marcada. Durante o processo de compilação e gravação do firmware na memória flash do ESP32, a Arduino IDE invoca o script em Python `esptool.py`.

2.2 Passo 2: Instalação do Driver USB-Serial

O ESP32 conecta-se ao computador através de um chip conversor serial. Identifique o chip da sua placa:

- **Silicon Labs CP2102/CP2104:** Caso o Windows não reconheça automaticamente, instale o driver oficial *CP210x Universal Windows Driver* (disponível no site da Silicon Labs).
- **WCH CH340:** Presente em placas alternativas, requer o instalador *CH341SER.EXE*.

Confirmação: Abra o **Gerenciador de Dispositivos** → expanda **Portas (COM e LPT)**. Deve constar uma porta ativa (ex.: `Silicon Labs CP210x USB to UART Bridge (COM3)`).

2.3 Passo 3: Adição do Repositório da Placa no Arduino IDE

1. Abra o **Arduino IDE** (versão 2.x recomendada).
2. Acesse **Arquivo** → **Preferências** (atalho `Ctrl + ,`).
3. No campo **URLs do Gerenciador de Placas Adicionais**, cole exatamente a URL oficial da Espressif:

   ```
   https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json
   ```

4. Clique em **OK**.
5. No menu lateral, acesse o **Boards Manager** (ícone de placa), digite `esp32` (desenvolvido por *Espressif Systems*) e clique em **Instalar**.

Tutoriais Complementares Indicados pelo Coordenação da Disciplina

Nos slides de aula da UD5, o Coordenação da Disciplina recomenda os seguintes tutoriais para apoio à instalação:

- Guia Fernando K: https://www.fernandok.com/2018/09/instalando-esp32-no-arduino-ide.html
- Guia CapSistema: https://capsistema.com.br/index.php/2021/02/26/programacao-da-placa-esp32-com-arduino-ide/

2.4 Passo 4: Seleção da Placa e Parâmetros de Upload

No menu superior **Ferramentas** (*Tools*):

- **Placa (Board):** Selecione `DOIT ESP32 DEVKIT V1` (ou `ESP32 Dev Module`).
- **Porta (Port):** Selecione a porta COM identificada no Gerenciador de Dispositivos.
- **Upload Speed:** `115200` bps (ou `921600` bps para gravação acelerada).
- **Flash Frequency:** `80MHz`.
- **Core Debug Level:** `None`.

3 Arquitetura de Hardware e Pinagem Segura

3.1 Pinout e Restrições Físicas do ESP32 (30 pinos)

O ESP32 DevKit V1 de 30 pinos possui limitações elétricas fundamentais que devem ser respeitadas para evitar queima da placa ou travamento de inicialização:

Pinos Proibidos para Relés: Strapping Pins e Input-Only

1. **Pinos de Strapping (GPIO 0, 2, 12 e 15):** São verificados pelo circuito interno do SoC durante o instante de ligamento/reset para definir a fonte de clock e o modo de boot (execução da flash ou modo download). Se um módulo relé estiver conectado a esses pinos, ele forçará o pino em nível alto ou baixo no boot, travando a inicialização do ESP32! **Nunca conecte os relés nos GPIOs 0, 2, 12 e 15.**
2. **Pinos Somente Entrada (GPIO 34, 35, 36/VP e 39/VN):** Não possuem transistores internos de driver de saída nem resistores de pull-up/pull-down. Funcionam apenas como entradas analógicas ou digitais.
3. **Pinos da Flash SPI (GPIO 6 a 11):** São conectados internamente ao chip de memória flash SPI. Qualquer conexão externa causará travamento imediato do microcontrolador.

3.2 A Lógica Active-LOW do Módulo Relé

Os módulos de relé da Eletrogate possuem entradas acopladas opticamente (fototransistor interno com LED infravermelho). O circuito é configurado em **Lógica Ativa em Nível Baixo (Active-LOW)**:

- `digitalWrite(PINO, LOW)` → O circuito drena corrente pelo catodo do optoacoplador, polarizando o transistor de driver. O relé **atraca** (o contato COM une-se ao contato NA), **acendendo a lâmpada**.
- `digitalWrite(PINO, HIGH)` → O LED do optoacoplador apaga, o transistor corta e a mola do relé retorna o contato para NF, **apagando a lâmpada**.

Boas Práticas de Programação no Setup

Para evitar que as lâmpadas pisquem ou todos os 4 relés atraquem no exato segundo em que o ESP32 é energizado, a inicialização no `setup()` deve ser feita obrigatoriamente nesta ordem:

```
digitalWrite(PINO_RELE, HIGH); // Garante estado DESLIGADO antes de habilitar
pinMode(PINO_RELE, OUTPUT);    // Configura pino como saida
digitalWrite(PINO_RELE, HIGH); // Reforco de seguranca
```

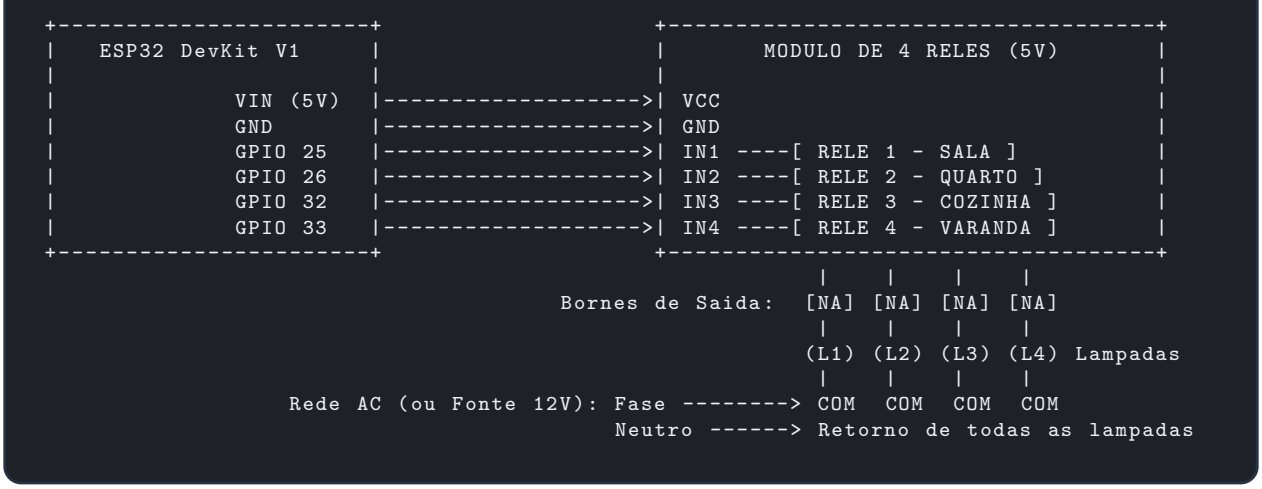
3.3 Tabela de Conexão Física de Bancada

Tabela 1: Mapeamento Seguro de Conexões entre ESP32 e Módulo de 4 Relés

Pino no Relé	Pino no ESP32	Função no Sistema	Justificativa Técnica
VCC	`VIN` (5V)	Alimentação das bobinas	Relés exigem 5V para atracamento confiável.
GND	`GND`	Terra comum do circuito	Referência de tensão unificada.
IN1	`GPIO 25`	Lâmpada 1 (Sala)	GPIO segura, saída digital nativa.
IN2	`GPIO 26`	Lâmpada 2 (Quarto)	GPIO segura, saída digital nativa.
IN3	`GPIO 32`	Lâmpada 3 (Cozinha)	GPIO segura, sem interferência no boot.
IN4	`GPIO 33`	Lâmpada 4 (Varanda)	GPIO segura, sem interferência no boot.

3.4 Diagrama de Conexões em Bloco (Bancada de Laboratório)

Esquemático de Ligações Físicas e Cargas de Potência



4 Roteiro Prático 1: Teste de Bancada e Acionamento Sequencial

O objetivo deste primeiro teste é verificar a integridade eletromecânica dos 4 relés, os LEDs indicadores do módulo e a resposta de depuração no Monitor Serial da Arduino IDE.

```
/*
 * Embedded Network Lab & ESP32 IoT Automation
 * Roteiro 1: Teste Sequencial do Modulo de 4 Reles
 * Plataforma: ESP32 DevKit V1
 */

#define RELAY_1 25
#define RELAY_2 26
#define RELAY_3 32
#define RELAY_4 33

#define RELAY_ON  LOW
#define RELAY_OFF HIGH

const int relayPins[4] = {RELAY_1, RELAY_2, RELAY_3, RELAY_4};
```

```
const char* lampNames[4] = {"Sala", "Quarto", "Cozinha", "Varanda"};

void setup() {
    Serial.begin(115200);
    delay(1000);
    Serial.println("\n=== IME - Redes I: Teste Inicial do Modulo de 4 Reles ===");

    for (int i = 0; i < 4; i++) {
        digitalWrite(relayPins[i], RELAY_OFF);
        pinMode(relayPins[i], OUTPUT);
        digitalWrite(relayPins[i], RELAY_OFF);
    }
    Serial.println("[OK] Reles inicializados em estado DESLIGADO (HIGH).");
}

void loop() {
    Serial.println("\n--- Iniciando Ciclo de Acionamento Sequencial ---");
    for (int i = 0; i < 4; i++) {
        Serial.printf("[ACIONANDO] Lampada %d (%s) no GPIO %d...\n", i+1, lampNames
            [i], relayPins[i]);
        digitalWrite(relayPins[i], RELAY_ON);
        delay(1500);

        Serial.printf("[DESLIGANDO] Lampada %d (%s)...\n", i+1, lampNames[i]);
        digitalWrite(relayPins[i], RELAY_OFF);
        delay(500);
    }

    Serial.println("[TESTE COLETIVO] Acionando todas as 4 lampadas...");
    for (int i = 0; i < 4; i++) digitalWrite(relayPins[i], RELAY_ON);
    delay(2000);

    Serial.println("[TESTE COLETIVO] Desligando todas as 4 lampadas...");
    for (int i = 0; i < 4; i++) digitalWrite(relayPins[i], RELAY_OFF);
    delay(3000);
}
```

Listing 1: Código de Teste Sequencial dos 4 Relés (`01_teste_reles.ino`)

5 Roteiro Prático 2: Servidor Web HTTP Wi-Fi (Dual Mode)

No contexto de Redes de Computadores, este experimento demonstra a implementação de um servidor de aplicação na camada de topo (HTTP/TCP na porta 80) sobre uma pilha TCP/IP completa no ESP32.

5.1 Topologia de Rede Resiliente: Modo Dual (STA + SoftAP)

Para garantir alta disponibilidade operacional e operação em campo sem roteador:

1. **Modo Station (STA):** O firmware tenta conectar-se à rede Wi-Fi local pré-configurada (`SUA_REDE_WIFI`) ou ao roteador móvel do operador.
2. **Fallback para Ponto de Acesso Próprio (SoftAP):** Caso a rede não esteja disponível após 15 tentativas, o ESP32 cria sua própria rede Wi-Fi local (`ESP32-IoT-Server`, senha `senha_ponto_acesso`). Basta conectar o celular nessa rede e acessar no navegador o endereço IP padrão `http://192.168.4.1`.

```
#include <WiFi.h>

// Parametros de conexao da rede local Wi-Fi
```

```
const char* sta_ssid     = "SUA_REDE_WIFI";
const char* sta_password = "";

// Ponto de Acesso Proprio para contingencia
const char* ap_ssid      = "ESP32-IoT-Server";
const char* ap_password  = "senha_ponto_acesso";

#define RELAY_ON  LOW
#define RELAY_OFF HIGH

const int relayPins[4] = {25, 26, 32, 33};
const char* lampNames[4] = {"Sala", "Quarto", "Cozinha", "Varanda"};
bool relayStates[4] = {false, false, false, false};

WiFiServer server(80);

void setRelay(int idx, bool on) {
    if (idx >= 0 && idx < 4) {
        relayStates[idx] = on;
        digitalWrite(relayPins[idx], on ? RELAY_ON : RELAY_OFF);
    }
}

void sendHtmlResponse(WiFiClient& client) {
    client.println("HTTP/1.1 200 OK\r\nContent-Type: text/html; charset=UTF-8\r\
        nConnection: close\r\n");
    client.println("<!DOCTYPE html><html><head><meta charset='UTF-8'>");
    client.println("<meta name='viewport' content='width=device-width, initial-
        scale=1.0'>");
    client.println("<style>body{font-family:sans-serif;background:#0f172a;color:#
        f8fafc;padding:20px;text-align:center;}");
    client.println(".card{background:#1e293b;padding:15px;margin:10px auto;max-
        width:380px;border-radius:10px;border:1px solid #334155;}");
    client.println(".btn{display:inline-block;padding:10px 24px;border-radius:6px;
        font-weight:bold;text-decoration:none;margin:4px;}");
    client.println(".btn-on{background:#0284c7;color:#fff;} .btn-off{background
        :#475569;color:#cbd5e1;}");
    client.println(".on{color:#4ade80;} .off{color:#f87171;}</style></head><body>")
        ;
    client.println("<h2>ESP32 IoT Automation Hub</h2><p>Controle de Iluminacao
        Residencial & Automacao Industrial</p>");

    for (int i = 0; i < 4; i++) {
        client.printf("<div class='card'><h3>Lampada %d - %s</h3>", i+1, lampNames[
            i]);
        if (relayStates[i]) {
            client.printf("<p class='on'>ESTADO: ACESA</p><a class='btn btn-off'
                href='/lamp%d/off'>Desligar</a>", i+1);
        } else {
            client.printf("<p class='off'>ESTADO: APAGADA</p><a class='btn btn-on'
                href='/lamp%d/on'>Ligar</a>", i+1);
        }
        client.println("</div>");
    }
    client.println("<br><a class='btn' style='background:#22c55e;' href='/all/on'>
        Ligar Todas</a> ");
    client.println("<a class='btn' style='background:#ef4444;' href='/all/off'>
        Desligar Todas</a>");
    client.println("</body></html>");
}

void loop() {
    WiFiClient client = server.available();
    if (!client) return;
```

```
55
56      String currentLine = "";
57      String requestHeader = "";
58      unsigned long timeout = millis() + 2000;
59
60      while (client.connected() && millis() < timeout) {
61          if (client.available()) {
62              char c = client.read();
63              requestHeader += c;
64              if (c == '\n') {
65                  if (currentLine.length() == 0) {
66                      if (requestHeader.indexOf("GET /lamp1/on") >= 0) setRelay(0,
                            true);
67                      else if (requestHeader.indexOf("GET /lamp1/off") >= 0) setRelay
                            (0, false);
68                      else if (requestHeader.indexOf("GET /lamp2/on") >= 0) setRelay
                            (1, true);
69                      else if (requestHeader.indexOf("GET /lamp2/off") >= 0) setRelay
                            (1, false);
70                      else if (requestHeader.indexOf("GET /lamp3/on") >= 0) setRelay
                            (2, true);
71                      else if (requestHeader.indexOf("GET /lamp3/off") >= 0) setRelay
                            (2, false);
72                      else if (requestHeader.indexOf("GET /lamp4/on") >= 0) setRelay
                            (3, true);
73                      else if (requestHeader.indexOf("GET /lamp4/off") >= 0) setRelay
                            (3, false);
74                      else if (requestHeader.indexOf("GET /all/on") >= 0) { for(int i
                            =0;i<4;i++) setRelay(i,true); }
75                      else if (requestHeader.indexOf("GET /all/off") >= 0) { for(int
                            i=0;i<4;i++) setRelay(i,false); }
76
77                      sendHtmlResponse(client);
78                      break;
79                  } else currentLine = "";
80              } else if (c != '\r') currentLine += c;
81          }
82      }
83      client.stop();
84  }
```

Listing 2: Trecho Principal do Servidor Web HTTP (`02_webserver_4_lampadas.ino`)

6 Roteiro Prático 3: Controle e Telemetria via Bluetooth Classic

Para controle local sem dependência de infraestrutura Wi-Fi, implementamos a comunicação sem fio via perfil serial Bluetooth Classic (SPP – *Serial Port Profile*).

6.1 Operação com o Aplicativo no Smartphone

Procedimento para conexão e operação via smartphone:

1. Instale no smartphone o aplicativo gratuito **Serial Bluetooth Terminal** (desenvolvido por Kai Morich).
2. Ligue o Bluetooth do celular e faça o pareamento com o dispositivo `ESP32-IoT-Server`.
3. No aplicativo, abra o menu lateral → selecione **Devices** → toque em `ESP32-IoT-Server` e toque no ícone de conexão.
4. Envie os comandos textuais em maiúsculo definidos na Tabela 2.

Tabela 2: Protocolo de Comandos Textuais via Bluetooth Serial

Comando (ASCII)	**Efeito Eletromecânico e Resposta do ESP32**
`L1 ON` / `L1 OFF`	Aciona ou desliga a Lâmpada 1 (Sala – GPIO 25)
`L2 ON` / `L2 OFF`	Aciona ou desliga a Lâmpada 2 (Quarto – GPIO 26)
`L3 ON` / `L3 OFF`	Aciona ou desliga a Lâmpada 3 (Cozinha – GPIO 32)
`L4 ON` / `L4 OFF`	Aciona ou desliga a Lâmpada 4 (Varanda – GPIO 33)
`ALL ON` / `ALL OFF`	Aciona ou desliga simultaneamente as 4 lâmpadas
`STATUS`	Retorna no terminal o estado detalhado de cada cômodo
`HELP`	Imprime na tela do smartphone a lista completa de comandos válidos

7 Guia de Troubleshooting: Resolução de Falhas em Bancada

1. **Falha na Gravação da Placa:**
   `A fatal error occurred: Failed to connect to ESP32: Timed out waiting for packet header`
   - *Diagnóstico:* O circuito de auto-reset do módulo não conseguiu aterrar o pino GPIO 0 no instante da sincronização serial do `esptool.py`.
   - *Solução Imediata:* No instante em que o console da IDE exibir os pontos de conexão (`Connecting.....____`), mantenha pressionado o botão físico **BOOT** da placa até aparecer a mensagem `Writing at 0x00010000... (5%)`. Em seguida, solte o botão.
   - *Solução por Hardware:* Caso o problema persista, conecte um capacitor eletrolítico de 10 μF entre os pinos **EN** e **GND**.
2. **Reinicialização Espontânea do ESP32 (*Brownout detector was triggered*):**
   - *Diagnóstico:* Cada bobina de relé consome cerca de 70 mA. Ao acionar os 4 relés somados à corrente de pico do transmissor Wi-Fi (> 250 mA), a corrente solicitada supera o limite da porta USB do computador ou a resistência interna de cabos longos, fazendo a linha de 5V cair abaixo de 4.5V.
   - *Solução:* Alimentar o borne VCC dos relés com uma fonte externa de 5V (2A), mantendo o fio de terra (**GND**) unificado com o GND do ESP32.
3. **Relés acionando de forma invertida ou ligando no boot:**
   - *Diagnóstico:* Módulo em lógica *Active-LOW*. A chamada de `pinMode(pino, OUTPUT)` por padrão assume nível zero, atracando os relés.
   - *Solução:* Sempre chamar `digitalWrite(pino, HIGH)` **antes** de chamar `pinMode(pino, OUTPUT)`.

8 Estrutura e Critérios do Relatório Técnico de Engenharia

Para fins de documentação formal de projetos de engenharia eletrônica e computação, a especificação técnica deve estruturar-se rigorosamente com os seguintes itens:

1. **Capa e Identificação Técnica:** Título formal do projeto, identificação institucional, autores e objetivos centrais do experimento.
2. **Introdução Teórica e Objetivos:** Fundamentos de sistemas embarcados IoT, microcontrolador ESP32, modelo cliente-servidor HTTP e protocolo Bluetooth SPP.
3. **Diagrama de Blocos e Topologia de Rede:** Esquema ilustrando a comunicação do celular com o ESP32 via Ponto de Acesso e o isolamento galvânico perante a carga de potência.

4. **Esquemático Elétrico de Conexões:** Tabela e diagrama de ligação dos pinos, demonstrando a proteção dos pinos de *strapping*.
5. **Código-Fonte Comentado:** Apresentação limpa dos arquivos `.ino` desenvolvidos e explicados bloco a bloco.
6. **Resultados Experimentais:** Capturas de tela da interface Web no smartphone, prints do console Serial e fotos da montagem física na bancada de laboratório.
7. **Conclusões e Lições Aprendidas:** Análise das dificuldades encontradas (lógica invertida, dimensionamento de corrente, interferências de rede) e propostas de expansão (ex.: integração com protocolo MQTT e dashboards em nuvem).

Documento técnico de engenharia de redes e sistemas embarcados. Códigos-fonte completos disponíveis na pasta `src/` e `c_sockets/` deste repositório.